

Async CV Traffic Pipeline

Technical Architecture Documentation

Producer-Consumer Design · Real-Time ANPR · DevOps Infrastructure

Project	Async Computer Vision Traffic Monitoring Pipeline
Version	1.0
Type	Microservice · Real-Time Computer Vision · ANPR
Stack	YOLOv26n, Redis, OpenCV/DSP, OCR, Python AsyncIO, Kubernetes
Focus	CCNA Networking + DevOps Engineering

1. Executive Summary

This document describes the complete architecture of a high-throughput, production-grade Automatic Number Plate Recognition (ANPR) system designed for multi-lane traffic monitoring. The system is engineered around a fundamental principle: real-time ingestion and compute-intensive inference cannot share the same execution thread without creating latency cascades that cause missed detections.

The solution is a Producer-Consumer architecture — three decoupled microservice stages separated by an asynchronous buffer. Each stage operates independently at its own optimal speed, making the overall system resilient to traffic bursts, hardware heterogeneity, and downstream slowdowns.

Core Engineering Problem Solved

At a busy 3-way intersection, 6 cars may enter the frame within 3 seconds. A naive linear pipeline would stall the camera feed while waiting for OCR to finish on the first car. This architecture solves that by separating Input Speed from Processing Depth — the camera always runs at 100% efficiency regardless of downstream inference time.

1.1 Architecture at a Glance

Layer	Stage 1	Buffer	Stage 2 + 3
Role	Edge Ingress (Producer)	Async Broker	Inference + Egress (Consumer)
Speed	30 FPS (camera rate)	Burst absorber	Variable (OCR-dependent)
Technology	YOLOv26n MOT	Redis / Python Queue	Custom YOLO + DSP + OCR
Output	Task + JPEG crop	Deduplicated task queue	Plate + speed + timestamp

Stage 1 Edge Ingress — The Producer

Real-time object detection, tracking, and snapshot event generation

2.1 Responsibility & Design Goal

Stage 1 is the gateway of the entire system. It receives the raw RTSP video stream from one or more IP cameras and runs continuously at the camera's native frame rate — typically 30 FPS — without any pause, stall, or backpressure from downstream stages. Its only job is to watch, detect, and fire off a single event per unique vehicle.

2.2 Core Technology: YOLOv26n with Multi-Object Tracking (MOT)

YOLOv26n is a lightweight variant of the YOLO (You Only Look Once) object detection family, optimized for low-latency inference on edge hardware such as NVIDIA Jetson or Intel NUC devices. It processes each frame and outputs bounding boxes, confidence scores, and class labels for every detected object.

Layered on top of detection is Multi-Object Tracking (MOT), which assigns a unique integer `track_id` to every vehicle and maintains that identity across consecutive frames even as the vehicle moves through the scene. Popular MOT algorithms used with YOLO include ByteTrack and StrongSORT.

Why MOT is essential:

- Without MOT, the same car would appear as a different detection in every frame, generating hundreds of redundant processing tasks.
- MOT stitches detections across time, allowing the system to treat a car as a single entity with a persistent identity.
- The `track_id` is the primary deduplication key used throughout the entire pipeline.

2.3 Speed Estimation via Vector Displacement

Since Stage 1 tracks each car across sequential frames, it has access to its positional history. Speed is estimated by computing the vector displacement — the pixel-space movement of the bounding box centroid — between frames and converting that to a real-world velocity using a known homography transformation (a mapping from pixel coordinates to meters, calibrated during camera installation).

Parameter	Detail
Unit of measurement	km/h or m/s (configurable)
Calibration requirement	Ground-truth reference distances in camera FOV

Parameter	Detail
Accuracy	~5-10% error at 30 FPS with stable tracking
Temporal smoothing	Moving average over last N frames recommended

2.4 The Snapshot Event

The Snapshot Event is the output contract of Stage 1. Rather than streaming raw video frames downstream (which would be enormously bandwidth-intensive), Stage 1 fires exactly one event per unique track_id the first time a vehicle is cleanly detected with sufficient confidence.

What the Snapshot Event Contains

- 1. Cropped vehicle image (JPEG or PNG, typically 224x224 or 416x416 pixels) — tightly bounded around the vehicle bounding box with a small padding margin.
- 2. Metadata payload: { track_id, estimated_speed_kmh, timestamp_utc, camera_id, frame_number, confidence_score }

This design is what enables the pipeline to remain non-blocking. Stage 1 never waits for a reply. It fires and forgets, pushing the event into the buffer and immediately returning to processing the next frame.

Layer 2 The Buffer — Asynchronous Broker

Shock absorber that decouples ingestion speed from inference speed

3.1 The Fundamental Problem It Solves

Stage 1 can generate events faster than Stage 2 can process them — especially during traffic bursts. Without a buffer, Stage 1 would either have to wait (blocking the camera feed) or drop events (missing cars). The Buffer Layer eliminates both failure modes.

3.2 The FIFO Queue

The queue is a First-In, First-Out data structure that stores Task objects — each containing the snapshot image and its associated metadata. Vehicles are processed in the order they were detected, which is important for timeline-accurate logging.

Implementation options:

- Redis Queue (recommended for production): Redis is an in-memory data store that supports native list operations (LPUSH / BRPOP) which behave as a queue. It is network-accessible, meaning Stage 1 (at the camera) and Stage 2/3 workers (in the cloud or a separate server) can communicate over TCP. Redis also supports persistence to disk, so tasks are not lost if a worker restarts.
- Python `asyncio.Queue` (for single-machine / development): A pure-Python in-process queue. Simpler to set up but does not scale across multiple machines or processes by default.

3.3 The Tracking Ledger (State Management)

The Tracking Ledger is a key-value store (typically a Redis SET or a Python dict) that records which `track_ids` have already had a task created. Before enqueueing a new snapshot, Stage 1 checks the ledger. If the `track_id` is already present, the event is discarded.

Why This Matters: The Traffic Jam Scenario

If a car is stuck in traffic and remains in the camera's field of view for 100 frames (over 3 seconds at 30 FPS), Stage 1 will detect and track it in every single frame. Without the ledger, 100 identical tasks would flood the queue. With the ledger, exactly one task is created for that car — the moment it was first cleanly detected.

3.4 Backpressure Handling & Auto-Scaling

Queue depth is a real-time health metric for the system. If Stage 2/3 workers are falling behind, the queue depth grows. The system exposes this metric to Kubernetes Horizontal Pod

Autoscaler (HPA) via a custom metrics adapter (e.g., KEDA — Kubernetes Event-Driven Autoscaling).

- Queue depth < 10 tasks: Single worker is sufficient.
- Queue depth 10-50 tasks: HPA scales to 2-4 worker pods.
- Queue depth > 50 tasks: HPA scales to maximum configured pod count.
- Queue depth returns to baseline: HPA scales down after a cool-down period.

Stage 2 Targeted Inference & Signal Conditioning

License plate localization and image preprocessing

4.1 Stage 2a — License Plate Detection (Custom YOLO)

Stage 2 receives a cropped vehicle image from the queue. The first sub-stage runs a second, specialized YOLO model whose sole purpose is to locate the license plate within the vehicle crop. This is not the same model as Stage 1 — it is a smaller, task-specific model trained exclusively on license plate bounding boxes.

Why a separate model?

- License plates are small, rectangular objects — typically occupying less than 5% of the vehicle image area. A general-purpose YOLO model is not optimized for this aspect ratio and scale.
- A dedicated model can be trained on thousands of annotated plate images across different lighting conditions, angles, and plate standards (EU, US, Middle Eastern, etc.).
- Smaller model = faster inference = higher throughput for Stage 2 workers.

The output of Stage 2a is a tight bounding box coordinate set: { x_min, y_min, x_max, y_max } defining the exact pixel region of the license plate within the vehicle crop.

4.2 Stage 2b — Digital Signal Processing (Noise Reduction)

Real-world license plate images suffer from multiple sources of degradation that reduce OCR accuracy: motion blur from vehicle movement, sensor noise in low-light conditions, glare from headlights, uneven shadow distribution, and lens distortion. Stage 2b applies a structured DSP preprocessing pipeline to correct these issues before passing the image to OCR.

DSP Pipeline (applied in sequence):

Step	Operation	Purpose
1. Grayscale conversion	Convert RGB to single-channel grayscale	Reduces dimensionality from 3 channels to 1, removing color noise and simplifying subsequent operations. Characters on plates are defined by luminance contrast, not color.
2. Bilateral filtering	Edge-preserving smoothing filter	Smooths pixel-level sensor noise and mild motion blur while preserving the sharp, high-contrast edges of alphanumeric characters. Unlike

Step	Operation	Purpose
		Gaussian blur, it does not smear character edges.
3. Adaptive thresholding	Local binarization (pixel-by-pixel)	Converts the grayscale image to pure black-and-white by computing a threshold value for each local pixel neighborhood. This corrects for uneven lighting — headlights illuminating one side, shadows on the other — producing a clean binary image ready for OCR.
4. Optional: Deskewing	Rotation correction via Hough Line Transform	If the camera angle causes the plate to appear tilted, this step detects the dominant line angle and rotates the image to horizontal alignment, reducing character segmentation errors.

Stage 3 Digitization & External Egress — The Consumer

OCR inference, data aggregation, and async downstream delivery

5.1 OCR Inference

Stage 3 receives the cleaned, binarized license plate image from Stage 2 and feeds it into an Optical Character Recognition engine. The OCR engine extracts the alphanumeric string (the plate number) from the image.

OCR engine options:

- Tesseract (open-source): Industry-standard, highly configurable. Can be tuned with a plate-specific whitelist of characters (e.g., only uppercase A-Z and 0-9) to reduce false positives. Runs on CPU.
- EasyOCR (open-source, deep learning): Better accuracy on degraded images due to deep learning backbone. Higher latency than Tesseract but GPU-acceleratable.
- Commercial APIs (Azure, AWS Rekognition): Highest accuracy, fully managed, but introduces network round-trip latency and per-call cost. Suitable for low-volume deployments.
- Custom CRNN (Convolutional Recurrent Neural Network): Maximum accuracy for a specific plate standard. Requires labeled training data but can achieve >99% accuracy on clean images.

5.2 Data Aggregation — Reuniting Stage 1 and Stage 3

The OCR output (plate number string) is a fragment of the full record. The complete data record is assembled by joining the OCR result with the metadata that was generated in Stage 1 and carried through the queue alongside the image:

Field	Source Stage	Description
plate_number	Stage 3 (OCR)	Alphanumeric string, e.g. '16ABK123'
speed_kmh	Stage 1 (MOT)	Estimated vehicle speed at time of snapshot
timestamp_utc	Stage 1	ISO 8601 timestamp of the snapshot event
camera_id	Stage 1	Identifier for the originating camera
track_id	Stage 1 (MOT)	Internal tracking identifier, used for deduplication
ocr_confidence	Stage 3 (OCR)	Confidence score 0.0-1.0 for the extracted plate string
processing_latency_ms	Stage 3	Total pipeline latency from snapshot to egress

5.3 Asynchronous External Egress

The final payload is dispatched to the secondary microservice (database, traffic authority system, parking management system, etc.) using Python's `asyncio` and `aiohttp` libraries. The use of asynchronous I/O is a critical design decision.

Why Async I/O for Egress?

If Stage 3 used synchronous (blocking) HTTP calls, a slow external database — even a brief 200ms timeout — would freeze the entire consumer worker while waiting for a response. With async I/O, Stage 3 fires the HTTP POST and immediately returns to processing the next task from the queue. Network latency in the external service has zero impact on internal pipeline throughput.

Egress protocol options:

- HTTP/REST POST: Simple, widely supported. Use for integrations with standard web APIs. Add retry logic with exponential backoff for resilience.
- Message Broker (Kafka, RabbitMQ, NATS): Preferred for high-throughput deployments. Decouples Stage 3 from the downstream consumer entirely, enabling fan-out to multiple subscribers (database, alerting system, dashboard) from a single publish event.
- gRPC: Lower overhead than REST for internal microservice communication. Requires proto schema definition but provides strong typing and bi-directional streaming.

6. Infrastructure & Network Topology

6.1 OSI Layer Reference for This System

Understanding which OSI layer each component operates at is essential for diagnosing performance issues, configuring firewalls, and planning network segmentation.

OSI Layer	Component	Technology	Engineering Function
Layer 2 / 3	Video network	VLAN tagging (802.1Q)	Isolates heavy RTSP multicast traffic from the corporate LAN. Prevents camera streams from congesting office network switches. Cameras, NVRs, and Stage 1 edge devices share a dedicated VLAN.
Layer 4	Video transport	UDP (RTP over RTSP)	RTSP video uses UDP for low-latency delivery. Dropped packets are tolerated — a single lost frame is preferable to buffering delays. UDP's connectionless nature also reduces TCP handshake overhead for continuous streams.
Layer 4	Data delivery	TCP (HTTP/HTTPS)	Final payload delivery uses TCP for guaranteed, ordered delivery. A missed plate detection record is unacceptable — unlike a dropped video frame, it cannot be recovered from context.
Layer 7	API gateway	REST / gRPC	External microservice handshake. TLS mandatory. API key or mTLS for authentication.

6.2 Kubernetes Orchestration

The pipeline is containerized using Docker and orchestrated with Kubernetes (K8s). This enables horizontal scaling, automated restarts on failure, rolling deployments, and resource quota enforcement.

Pod topology:

- Stage 1 pods: DaemonSet or StatefulSet. One pod per camera. Pinned to the edge node where the camera RTSP stream is accessible. Cannot be arbitrarily scaled — each pod maps to a physical camera.

- Stage 2/3 worker pods: Deployment with HPA. These are stateless consumers that read from the queue. They can be scaled horizontally from 1 to N replicas based on queue depth. Each replica independently consumes tasks and processes them in parallel.
- Redis pod: StatefulSet with persistent volume. Single primary with optional replica for read scaling. Consider Redis Cluster for very high throughput deployments.

6.3 DevSecOps Practices

Security must be embedded in the pipeline from the start, not bolted on after deployment. The following practices apply:

- Container image scanning: All Docker images are scanned for known CVEs (Common Vulnerabilities and Exposures) using Trivy or Snyk before being pushed to the registry. Critical vulnerabilities block deployment.
- Secrets management: RTSP credentials, API keys, and database passwords are never hardcoded in Dockerfiles or Kubernetes manifests. They are injected at runtime via Kubernetes Secrets (base64-encoded) or a dedicated secrets manager such as HashiCorp Vault or AWS Secrets Manager.
- Network policies: Kubernetes NetworkPolicies restrict pod-to-pod communication. Stage 1 pods can only reach the Redis queue. Stage 2/3 workers can read from Redis and write to the external API. No pod has unrestricted egress.
- Image provenance: Use a private container registry with image signing (Cosign / Notary) to prevent supply-chain attacks.

7. Burst Handling — The 6-Car Scenario

This section traces exactly what happens when 6 vehicles enter the intersection simultaneously within a 3-second window — the stress case the architecture was designed to handle.

Time	Actor	Event
t=0s	Stage 1	6 cars enter frame. YOLOv26n detects all 6 and assigns track_ids 101-106 within the first 100ms.
t=0.1s	Stage 1	Speed vectors computed for all 6. 6 Snapshot Events generated. 6 tasks pushed to Redis queue. Ledger updated with IDs 101-106. Stage 1 returns to 30 FPS.
t=0.1s	Buffer	Redis queue depth: 6. HPA threshold not yet crossed. Single Stage 2/3 worker begins processing task for track_id 101.
t=0.8s	Stage 1	Same 6 cars still in frame. Track IDs 101-106 already in ledger. Zero new tasks generated. Camera feed continues uninterrupted.
t=1.0s	Stage 2/3	Track 101 OCR complete. Payload dispatched async. Worker immediately pops task 102.
t=2.5s	Stage 2/3	All 6 tasks processed and egressed. Zero missed detections. Total pipeline latency per vehicle: ~400ms.

Key Insight

Stage 1 completed its work in 100ms for all 6 cars. Stage 2/3 took 2.5 seconds to finish all 6. This 25x speed mismatch is invisible to Stage 1 — it is entirely absorbed by the buffer. Without the buffer, Stage 1 would stall for 2.5 seconds and miss any cars that passed through the intersection during that window.

8. Geo-Distributed Deployment — WAN Latency Challenge

Deploying Stage 1 at the camera site (on-premises edge) and the FIFO Queue in the cloud introduces Wide Area Network (WAN) latency between them. This is a critical design consideration because the Snapshot Event includes a JPEG image that may be 50-200 KB in size. On a congested WAN link with 80ms RTT, transferring 100 KB at 10 Mbps takes ~80ms — before accounting for TCP congestion control, retransmits, or TLS handshake overhead.

8.1 Problem Statement

- RTSP video streams should never traverse the WAN — they are consumed locally by Stage 1.
- Only Snapshot Events (small crops + metadata) need to cross the WAN to reach the cloud queue.
- However, even small transfers can time out if the WAN link is congested or the RTT is high (>200ms for intercontinental links).

8.2 Mitigation Strategies

Strategy 1: Edge-Side Store-and-Forward Buffer

Deploy a local Redis instance (or a lightweight message queue like NATS or MQTT broker) at the camera site. Stage 1 pushes tasks to the local queue immediately — this operation is near-zero latency on LAN. A separate forwarding agent reads from the local queue and asynchronously transfers tasks to the cloud queue, handling retries, backpressure, and reconnection logic independently of Stage 1.

Strategy 2: QUIC Transport Protocol

Replace TCP with QUIC (HTTP/3) for the WAN transfer of Snapshot Events. QUIC provides multiplexed streams, built-in TLS, and dramatically faster connection establishment (0-RTT or 1-RTT vs TCP's 3-way handshake + TLS). Most critically, QUIC eliminates Head-of-Line Blocking — a single lost UDP packet does not stall the entire connection, unlike TCP streams. This is particularly beneficial on lossy mobile or satellite WAN links.

Strategy 3: Regional Queue Replicas

Deploy Redis instances in each geographic region (e.g., eu-west-1, ap-southeast-1). Stage 1 writes to the nearest regional queue with ~5ms latency. A replication layer (Kafka MirrorMaker, Redis replication, or a custom forwarder) synchronizes tasks between regional queues and the central processing cluster. Stage 2/3 workers in each region process their local queue, reducing WAN dependency entirely for the inference path.

Strategy 4: Image Compression Optimization

Reduce the payload size of Snapshot Events before WAN transfer. Apply progressive JPEG compression targeting 85% quality (imperceptible quality loss for OCR purposes, ~60% size reduction vs. uncompressed). Consider WebP format for additional 30% size savings over JPEG. For the metadata portion, use MessagePack or Protocol Buffers instead of JSON to reduce serialization overhead.

Strategy	Latency Reduction	Complexity	Best For
Store-and-forward edge buffer	Eliminates WAN from critical path	Medium	Unreliable WAN links
QUIC transport	30-50% faster connection setup	Medium	High-latency WAN (>80ms)
Regional queue replicas	Near-zero WAN latency	High	Multi-region deployments
Payload compression	Reduces transfer time 40-70%	Low	Bandwidth-constrained links

9. Full Technology Stack Reference

Component	Technology	Notes
Object detection	YOLOv26n	Lightweight, edge-optimized variant. Runs on NVIDIA Jetson, Intel NUC, or GPU server.
Multi-object tracking	ByteTrack / StrongSORT	Associates detections across frames. Produces persistent track_id per vehicle.
Message queue	Redis (LPUSH/BRPOP)	FIFO semantics, network-accessible, persistent. KEDA adapter for K8s HPA integration.
License plate detector	Custom YOLOv8n/v5	Trained on plate bounding boxes. Input: vehicle crop. Output: plate bbox coordinates.
Image preprocessing	OpenCV	Grayscale, bilateral filter, adaptive threshold, optional deskew.
OCR engine	Tesseract / EasyOCR / CRNN	Character whitelist A-Z 0-9. Confidence threshold filters low-quality reads.
Async I/O	Python asyncio + aiohttp	Non-blocking HTTP egress. Retry with exponential backoff.
Containerization	Docker	Multi-stage builds. Non-root user. Trivy/Snyk CVE scanning in CI.
Orchestration	Kubernetes (K8s)	HPA via KEDA (queue-depth metric). NetworkPolicies. Secrets via K8s Secrets or Vault.
Video transport	RTSP over UDP (RTP)	Local VLAN only. Never traverses WAN. H.264 or H.265 encoded stream.
Data transport	HTTP/REST or Kafka	TCP, TLS 1.3. Payload: JSON or MessagePack. Async non-blocking.
Network segmentation	VLAN 802.1Q	Dedicated VLAN for camera traffic. Firewall rules enforce inter-VLAN access control.
Secrets management	HashiCorp Vault / K8s Secrets	RTSP credentials, API keys. Never in Dockerfiles or manifests.

10. Future Improvements & Open Engineering Questions

10.1 Short-Term Enhancements

- Re-ID across cameras: Extend the tracking ledger to share track context across multiple camera feeds, enabling a single vehicle to be tracked as it moves through multiple zones.
- Plate validation: After OCR, apply a country-specific regex pattern (e.g., Algerian plates follow a specific format) to validate the extracted string and reject garbage reads before they reach the database.
- Night mode switching: Detect ambient light levels and switch the DSP pipeline to an IR-optimized preprocessing chain when night conditions are detected.
- Edge ML acceleration: Deploy TensorRT-optimized ONNX models on NVIDIA Jetson to reduce Stage 1 inference latency below 5ms per frame, enabling support for 60+ FPS cameras.

10.2 Open Architecture Questions

- WAN latency mitigation: If deploying Stage 1 at the camera site and the queue in the cloud, which combination of edge buffering + transport protocol (QUIC vs TCP) + payload compression achieves sub-100ms end-to-end latency on a 50 Mbps WAN link with 40ms RTT?
- Queue technology at scale: At what queue depth and event rate does Redis become a bottleneck, and what is the migration path to Apache Kafka for higher throughput?
- Multi-region consistency: If processing tasks are distributed across regional queues, how do you guarantee that a vehicle crossing a regional boundary is not processed twice and its full journey is assembled correctly in the central database?
- Privacy compliance: In jurisdictions where GDPR or similar regulations apply, what is the minimum retention window for JPEG crops and how is the right-to-erasure request propagated through a distributed queue?

End of Document